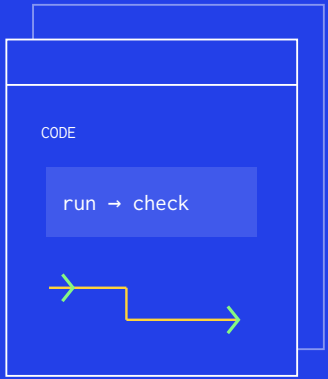
WEEK 2 / LAB 4 / TEAM WORK

MLOps: Machine Learning Operations

Serve the model. Fail honestly.

Expose the evaluated bike-demand model through a validated API and verify it in the container. Needs the Lab 3 image on main.



Done means checked behaviour, not a running process

Three outcomes you can demonstrate from the team repository.

1 **Serve**

Load the saved, evaluated model at startup and return its actual version. Never train inside the service.

2 **Validate**

Reject missing fields, wrong types and out-of-range values before inference.

3 **Verify**

Automated tests cover normal, invalid and missing-model cases, and they run inside the container.

Who does what

The split is yours; every member owns something and reviews something.

INDIVIDUAL

One PR each

One area each: validation, load + health/predict, tests, or container + README.

TEAM

One service

Agree contract, merge order and the final container check.

REVIEW

Another member

Break a PR you did not write: one bad request, one missing model.

Individual report reports/week-2.md covers Labs 3–4 in your personal repository.

Add these to the team repository

Paths are the contract; contents are your design.

REQUIRED

Reviewed PRs into main. Declare new dependencies in pyproject.toml and keep uv.lock in step.

TEAM REPOSITORY PATHS

```
src/bike_demand/api.py
tests/test_api.py
Dockerfile
pyproject.toml uv.lock
README.md
your team's own report
```

README: how to run and test the service. Report: your team chooses its format; add a Lab 4 section.

Two endpoints, three outcomes

Both share one model-availability state.

GET /health

200 or 503

200 only when a usable model is loaded; otherwise 503.

POST /predict

200 + version

JSON with a finite non-negative prediction and the loaded model_version.

OTHERWISE

4xx or 503

Invalid input gets a 4xx; valid input without a model gets 503. Never a fake prediction.

model_version comes from the artifact's own metadata, not a hard-coded string.

Input contract: twelve fields

One flat JSON object with the same fields used in training.

RANGES

Weather values are normalised, not Celsius or percentages. cnt, casual and registered are never accepted as inputs.

REQUEST FIELDS

```
season 1-4 yr 0-1
mnth 1-12 hr 0-23
holiday 0-1 weekday 0-6
workingday 0-1 weathersit 1-4
temp atemp hum windspeed
finite numbers in 0..1
```

Decide and document how extra fields are handled.

Tools you will meet

Lecture 4 covered API concepts; these are this lab's concrete tools.

BUILD

FastAPI + Pydantic

Routes, request models with field constraints, and an OpenAPI page at /docs.

TEST

pytest + TestClient

Call the app without a server; start each case in the state it tests.

RUN

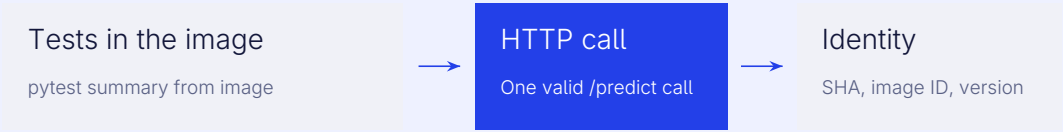
uvicorn + docker

Listen on 0.0.0.0 and publish a port to reach it.

Read the official documentation for each; choosing how to use them is the lab.

Prove it where it will run

Evidence comes from the built image, not from your laptop environment.



`A passing local pytest does not show the container tests ran.`

Keep raw logs your report can link to; at most three screenshots.

m2 and your individual report

One team checkpoint for Labs 3–4; one personal report.

TEAM

Annotated tag m2

On main after review and checks; submit tag link and a short note.

PERSONAL

reports/week-2.md

Your contribution, one decision, one blocker, assistance used.

RULES

Honest and on time

Personal main, no PR or CI. Sunday 23:59. 10% of course.

Incomplete work is reported as incomplete; never invent a result or a passing check.

EXIT CHECK

A usable service tells the truth when it fails.

Next, Lab 5: local deployment, structured logs and serving-version checks.